MCA Semester 1	Subject : Advanced Data Structures Lab
Name : Mukund Gangurde	Topic: Traversals 1. Depth-First-Traversal 2. Breadth-First-Traversal
Roll No. : MCA2511	Date : 03-12-2025

1. Depth-First-Traversal

Code:

```
class DFT
{
        int[][] adj;                    //Adjacency Matrix for growth
        boolean[] visited;      //Track visited status odf each vertex
        int[] stack;                    //Array-based stack
        int tos;            //Top of the stack

        //Constructor to initialize the graph
        public DFT(int v)
        {
                adj = new int[v][v];
                visited = new boolean[v];
                stack = new int[v];
                tos = -1;
        }//end of DFT

        //Add an edge to the graph
        public void addEdge(int src,int dest)
        {
                adj[src][dest] = 1;       //Add Edge
                adj[dest][src] = 1;       //Remove this for directed graph
        }//end of addEdge

        //DFT Traversal
        public void performDFT(int x)
        {
                push(x);          //Push starting node on the stack
                System.out.print("Depth-First Traversal: ");
                while(tos!=-1)  //While the stack not empty
                {
                        int curr = pop();          //Pop the tos
                        if(!visited[curr])
                        {
                                visited[curr] = true;      //Mark the node visited
```

```
                        System.out.print(curr + " ");

                        //Push all unvisited neighbours on to the stack
                        for(int i = 0; i < visited.length ; i++)
                        {
                                if(adj[curr][i] == 1 && !visited[i])
                                {
                                        push(i);
                                }//end of inner if
                        }//end of for
                }//end of outer if
        }//end of while
}//end of performDFT

//Push
public void push(int node)
{
        stack[++tos] = node;
}

//Pop
public int pop()
{
        return stack[tos--];
}

public static void main(String[] args)
{
        DFT g = new DFT(5);

        //Add edges
        g.addEdge(0,1);
        g.addEdge(0,2);
        g.addEdge(1,3);
        g.addEdge(1,4);
        g.addEdge(2,3);
        g.addEdge(3,4);

        //Perform DFT
        g.performDFT(1);

        g.addEdge(1,4);
        g.addEdge(1,3);
        g.addEdge(2,3);
        g.addEdge(2,4);
        g.addEdge(3,4);
```

```
            g.performDFT(1);


        }//end of psvm
}//end of DFT
```

Output:

```
//Add edges
g.addEdge(0,1);
g.addEdge(0,2);
g.addEdge(1,3);
g.addEdge(1,4);
g.addEdge(2,3);
g.addEdge(3,4);

//Perform DFT
g.performDFT(1);
```

```
A:\MCA2511\DS_LAB>javac 20DFT.java

A:\MCA2511\DS_LAB>java DFT
Depth-First Traversal: 1 4 3 2 0
```

```
g.addEdge(1,4);
g.addEdge(1,3);
g.addEdge(2,3);
g.addEdge(2,4);
g.addEdge(3,4);
//Perform DFT
g.performDFT(1)
```

```
A:\MCA2511\DS_LAB>javac 20DFT.java

A:\MCA2511\DS_LAB>java DFT
Depth-First Traversal: 1 4 3 2
```

2. Breadth-First Traversal.

Code:

```
class BFT
{
	int[][] adj;		//Adjacency Matrix For Graph
	boolean[] visited;	//Track Visited Status
	int[] queue;		//Array Based Queue
	int front, rear;

	//Constructor
	public BFT(int v)
	{
		adj = new int[v][v];
		visited = new boolean[v];
		queue = new int[v];
		front = -1;
		rear = -1;
	}//end of constructor

	//Add an edge to the graph
	public void addEdge(int src, int dest)
	{
		adj[src][dest] = 1;
		adj[dest][src] = 1;
	}//end of addEdge

	//Perform BFT
	public void performBFT(int x)
	{
		Enqueue(x);	//Enqueue The Starting Node
		visited[x] = true;

		System.out.print("Breadth First Traversal: ");
		while(front!=-1)
		{
			int curr = Dequeue();  //Dequeue
			System.out.print(curr + " ");

			//Enqueue Those Neighbout Of Curr That Are Not Visited
			for(int i=0; i<adj.length; i++)
			{
				if(adj[curr][i] == 1 && !visited[i])
				{
					Enqueue(i);
					visited[i] = true;
```

```
                                }//end of if
                        }//end of for
                }//end of while
        }//end of performBFT

        //Enqueue
        public void Enqueue(int node)
        {
                if(front == -1)
                {
                        front++;
                }
                queue[++rear] = node;
        }//end of Enqueue

        //Dequeue
        public int Dequeue()
        {
                int tmp = queue[front];
                if(front == rear)
                {
                        front = -1;
                        rear = -1;
                }
                else
                {
                        front++;
                }
                return tmp;
        }//end of Dequeue

        //Main
        public static void main(String[] args)
        {
                BFT g = new BFT(7);
                //Add edges
                g.addEdge(0,1);
                g.addEdge(0,2);
                g.addEdge(1,3);
                g.addEdge(2,4);
                g.addEdge(2,6);
                g.addEdge(3,5);
                g.addEdge(4,5);
                g.performBFT(0);
        }//end of psvm
}//end of BFT
```

Output:

```
A:\MCA2511\DS_LAB>javac 20_1BFT.java

A:\MCA2511\DS_LAB>java BFT
Breadth First Traversal: 0 1 2 3 4 6 5
```

```
g.addEdge(2,4);
g.addEdge(1,5);
g.addEdge(0,5);
g.addEdge(2,3);
g.addEdge(3,5);
g.performBFT(0);
```

```
A:\MCA2511\DS_LAB>javac 20_1BFT.java

A:\MCA2511\DS_LAB>java BFT
Breadth First Traversal: 0 5 1 3 2 4
```